

Accounting Analytics with Python

Analytics Powered by Large Language Models (LLM)

Sharif Islam, DBA, CPA, CMA

2026-07-14

Table of contents

Preface	1
About the Author	3
Acknowledgement	5
I. Part I: Foundations	7
1. Introduction to Accounting Analytics	9
1.1. What Is Accounting Analytics?	9
1.2. Why Now? The Shift Toward Data-Driven Accounting . . .	10
1.3. Accounting Analytics vs. Traditional Accounting and Auditing	11
1.4. The Four Types of Analytics	12
1.5. The Accounting Analytics Workflow	13
1.6. Where Accounting Analytics Shows Up in Practice	14
1.7. Careers and Certifications	14
1.8. A Note on Tools: Why Python and Quarto?	15
1.9. Chapter Summary	16
1.10. Discussion Questions	16
1.11. Exercises	17
2. Python and Quarto Environment Setup	19
2.1. Why This Chapter Matters	19
2.2. The Three Tools You Need	20
2.3. Step 1: Install Python	20
2.4. Step 2: Install Quarto	21

Table of contents

2.5. Step 3: Install a Code Editor	22
2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document	23
2.6.1. 1. A Plain Python Script (.py)	24
2.6.2. 2. A Jupyter Notebook (.ipynb)	24
2.6.3. 3. A Quarto Document (.qmd)	26
2.7. Your First Quarto Document	29
2.8. Rendering This Book's Project (HTML and PDF)	32
2.9. Installing the Python Packages You'll Need	33
2.10. Organizing Your Project Folder	34
2.11. Troubleshooting Common Setup Issues	36
2.12. Chapter Summary	36
2.13. Exercises	37
3. Data Fundamentals for Accountants	39
3.1. Why Data Structure Matters Before You Start Coding . . .	39
3.2. Tidy Data Principles	40
3.3. Levels of Measurement	41
3.4. Data Types You'll Encounter	43
3.4.1. A Special Note on Dates	43
3.5. Core Accounting Data Structures	44
3.5.1. Chart of Accounts (COA)	45
3.5.2. General Ledger (GL)	45
3.5.3. Subsidiary Ledgers	45
3.5.4. Trial Balance	45
3.6. Transaction-Level vs. Summarized Data	46
3.7. Common Data Quality Issues in Accounting Data	46
3.8. A Preview Look at a Real Dataset	47
3.9. Chapter Summary	49
3.10. Discussion Questions	50
3.11. Exercises	50

Table of contents

II. Part II: Core Data Skills	51
4. Data Wrangling with Pandas	53
5. Exploratory Data Analysis for Financial Data	55
6. Data Visualization for Accounting	57
III. Part III: Applied Accounting Analytics	59
7. Financial Statement Analysis with Python	61
8. Audit Analytics	63
9. Fraud Detection and Anomaly Detection	65
10. Revenue Recognition and Transaction-Level Analysis	67
11. Tax Analytics	69
IV. Part IV: Predictive Methods and GenAI	71
12. Introduction to Predictive Modeling	73
13. Generative AI in Accounting	75
V. Part V: Communication and Capstone	77
14. Automated Reporting and Dashboards with Quarto	79
15. Data Ethics, Governance, and Reproducibility	81
16. Capstone Project	83

Table of contents

References	85
Appendices	87
A. A Primer on Python	87
B. Financial Statement Basics	89
C. An Introduction to EDGAR Database	91

Preface

Welcome to *Accounting Analytics with Python*. This book is intended for accounting undergraduate students. However, students from other disciplines such as Finance are very likely to be benefited from using the book.

About the Author

Sharif Islam, DBA, CPA, CMA is an Associate professor in School of Accountancy in Southern Illinois University Carbondale (SIUC). He is a licensed CPA in Illinois and a Certified Management Accountant (CMA). He teaches Advanced Cost Accounting, Auditing, Accounting Information Systems, Machine Learning, and Analytics for Accounting Data. He earned his doctorate from Louisiana Tech University. His manuscripts are selected for “Best Research Paper Award” in several conferences of American Accounting Association (AAA). His research also got 2024 “Notable Contribution to the Literature Award” by AIS section of AAA. He published research in *Accounting Horizons*, *Journal of Accounting and Public Policy*, *Journal of Information Systems*, *Advances in Accounting*, *Journal of Emerging Technologies in Accounting*, *Issues in Accounting Education*, and *Managerial Auditing Journal*. His research interests lie at the intersection of Accounting and Data Science.

Acknowledgement

To prepare the book, I took help from many sources on the internet and published materials. Many of them are cited in the book. I acknowledge the contribution of all of those resources that help me to prepare the book for the students.

Part I.

Part I: Foundations

1. Introduction to Accounting Analytics

i Learning Objectives

By the end of this chapter, you should be able to:

- Define accounting analytics and distinguish it from traditional accounting and auditing
- Explain why data analytics has become central to accounting practice
- Describe the four major types of analytics (descriptive, diagnostic, predictive, prescriptive) and give an accounting example of each
- Outline the typical accounting analytics workflow, from raw data to a decision
- Identify career paths and professional certifications related to accounting analytics

1.1. What Is Accounting Analytics?

Accounting has always been about data. Every transaction a business records — a sale, a payment, a depreciation entry — becomes a data point that eventually rolls up into financial statements. What has changed in the

1. Introduction to Accounting Analytics

last two decades is not *that* accountants work with data, but the *volume*, *speed*, and *tools* available for working with it.

Accounting analytics is the application of data analysis techniques — statistical, visual, and computational — to accounting and financial data in order to support decisions in areas such as financial reporting, auditing, tax, and management accounting. It sits at the intersection of three fields:

- **Accounting knowledge** — understanding what the numbers mean (revenue recognition, internal controls, GAAP/IFRS rules)
- **Data skills** — the ability to collect, clean, and analyze data (which is where Python comes in)
- **Business judgment** — knowing what question is worth asking and what a result actually implies for a decision

A useful way to think about it: traditional accounting produces the numbers; accounting analytics asks *what the numbers are telling us* and *what we should do about it*.

1.2. Why Now? The Shift Toward Data-Driven Accounting

Several forces have pushed analytics from a “nice to have” to a core accounting skill:

1. **Data volume.** Modern ERP systems (SAP, Oracle, NetSuite) log every transaction electronically. A mid-sized company can generate millions of journal entries a year — far more than any auditor or accountant could review manually.
2. **Regulatory and audit expectations.** Standard-setters and regulators (including the PCAOB) increasingly expect auditors to test *entire populations* of transactions rather than small manual samples, which requires analytics tools.

1.3. Accounting Analytics vs. Traditional Accounting and Auditing

- 3. **Automation of routine work.** Many manual bookkeeping and reconciliation tasks are increasingly automated, shifting accountants' time toward analysis, interpretation, and communication of results.
- 4. **The rise of accessible tools.** Python, R, and business intelligence tools (Power BI, Tableau) have made analytics accessible to accountants without a computer science background — you don't need to be a software engineer to benefit from these tools.
- 5. **CPA Exam evolution.** The AICPA has incorporated data analytics content into the CPA Exam Blueprint, particularly in the Business Analysis and Reporting (BAR) discipline, signaling that the profession considers this a core competency, not an elective skill.

 Connect to Practice

Throughout this book, look for “Connect to Practice” boxes like this one, which tie a concept back to how it’s used in real accounting and audit work.

1.3. Accounting Analytics vs. Traditional Accounting and Auditing

It’s worth being precise about how this course differs from other accounting courses you’ve taken:

	Traditional Accounting/Auditing	Accounting Analytics
Primary tool	Spreadsheets, manual ledgers	Python, SQL, visualization tools
Sample size	Often a sample of transactions	Frequently the <i>entire population</i>

1. Introduction to Accounting Analytics

	Traditional Accounting/Auditing	Accounting Analytics
Focus	Was this recorded correctly?	What patterns, risks, or insights exist in the data?
Output	Journal entries, financial statements, audit opinions	Dashboards, models, flagged anomalies, predictions
Skillset emphasis	Rules and standards (GAAP/IFRS)	Rules and standards <i>plus</i> coding, statistics, visualization

This isn't a replacement for the accounting knowledge you've built in other courses — it's a set of tools that make that knowledge more powerful and scalable.

1.4. The Four Types of Analytics

A common framework in the analytics field breaks analysis into four types, moving from “what happened” to “what should we do.” Every technique in this book fits into one of these categories.

1. **Descriptive analytics** — *What happened?* Summarizing past data. Example: calculating total revenue by month, or the average days-sales-outstanding over the last year.
2. **Diagnostic analytics** — *Why did it happen?* Investigating causes and relationships. Example: determining why gross margin declined in Q3 by breaking revenue and cost data down by product line.
3. **Predictive analytics** — *What is likely to happen?* Using historical data to forecast. Example: predicting which customers are likely to become uncollectible accounts receivable.

1.5. The Accounting Analytics Workflow

4. **Prescriptive analytics** — *What should we do?* Recommending an action based on analysis. Example: recommending which vendors to flag for additional audit procedures based on a fraud-risk score.

Most introductory accounting analytics work — including much of this book — lives in the descriptive and diagnostic space, with an introduction to predictive methods later in the book (Chapter 12). Prescriptive analytics tends to require more advanced modeling and is largely beyond the scope of an undergraduate course, but it’s useful to know where the field is heading.

1.5. The Accounting Analytics Workflow

Most accounting analytics projects — whether a homework assignment in this course or a real audit engagement — follow a similar sequence:

1. **Define the question.** What decision or risk are we trying to inform? (e.g., “Are there unusual journal entries near period-end that warrant investigation?”)
2. **Obtain the data.** Extract data from a source system — a general ledger export, a CSV file, a database query.
3. **Clean and prepare the data.** Real data is messy: missing values, inconsistent formats, duplicate entries. This step often takes the most time in practice.
4. **Explore and analyze.** Apply descriptive statistics, visualizations, or models to look for patterns.
5. **Interpret results in context.** A statistical pattern is not automatically meaningful — this is where accounting judgment matters most.
6. **Communicate findings.** Produce a report, dashboard, or memo that a non-technical stakeholder (a partner, a controller, a client) can act on.

1. Introduction to Accounting Analytics

This book is structured around this workflow. Chapters 2–3 build foundational tools and data literacy; Chapters 4–6 cover steps 2–4 in depth; Chapters 7–11 apply the workflow to specific accounting domains (financial statements, audit, fraud, revenue, tax); and later chapters extend into predictive methods, GenAI, and communicating results.

1.6. Where Accounting Analytics Shows Up in Practice

A few examples of accounting analytics in action:

- **External audit** — Using full-population testing and anomaly detection to identify unusual journal entries for further audit procedures, rather than testing a small manual sample.
- **Internal audit** — Continuous monitoring dashboards that flag control violations or policy exceptions as they occur, not just at year-end.
- **Financial planning & analysis (FP&A)** — Building forecasting models and variance-analysis dashboards to support budgeting decisions.
- **Forensic accounting** — Applying techniques like Benford’s Law (covered in Chapter 8) to detect potential financial statement fraud.
- **Tax** — Analyzing large transactional datasets to identify tax planning opportunities or compliance risks across jurisdictions.

1.7. Careers and Certifications

Accounting analytics skills are increasingly valued across public accounting, industry, and government. A few relevant credentials and pathways worth knowing about as you plan your career:

1.8. A Note on Tools: Why Python and Quarto?

- **CPA (Certified Public Accountant)** — increasingly incorporates data analytics content, especially through the BAR discipline
- **CISA (Certified Information Systems Auditor)** — relevant for IT audit and control-focused roles
- **Data-analytics-specific certificates** offered by the AICPA and various universities

You don't need to become a data scientist to benefit from this course — the goal is *analytics literacy*: being comfortable enough with data tools that you can perform basic analyses yourself, and knowledgeable enough to critically evaluate analyses done by others (including, increasingly, by AI tools).

1.8. A Note on Tools: Why Python and Quarto?

This book uses **Python** as its programming language and **Quarto** as its authoring and reporting tool, for a few reasons that matter to your future career:

- Python is one of the most widely used languages in data analytics and is free and open-source, meaning you can continue using it after graduation without needing a paid license.
- The pandas library (introduced in Chapter 4) is purpose-built for exactly the kind of tabular, spreadsheet-like data accountants work with every day.
- Quarto allows you to combine narrative writing, code, and output (tables, charts) into a single reproducible document — which mirrors how analytics work is increasingly delivered in practice, as a reproducible report rather than a static spreadsheet.

You'll get hands-on with both tools starting in Chapter 2.

1. Introduction to Accounting Analytics

1.9. Chapter Summary

- Accounting analytics combines accounting knowledge, data skills, and business judgment to support decisions.
- The shift toward analytics is driven by data volume, regulatory expectations, automation, and accessible tools.
- Analytics can be descriptive, diagnostic, predictive, or prescriptive — most of this course focuses on the first two, with an introduction to the third.
- A consistent workflow (define, obtain, clean, analyze, interpret, communicate) underlies most accounting analytics projects, and structures the rest of this book.
- These skills map directly onto current and evolving areas of accounting practice, including the CPA Exam.

1.10. Discussion Questions

1. Choose one accounting role (e.g., external auditor, tax accountant, FP&A analyst). How might accounting analytics change the day-to-day work of someone in that role over the next five years?
2. Think of a business decision (not necessarily accounting-related) you or your family have made recently. Which of the four analytics types (descriptive, diagnostic, predictive, prescriptive) best describes the thinking that went into it?
3. Why do you think regulators like the PCAOB have pushed auditors toward analyzing full populations of transactions rather than samples? What are the potential benefits and risks of this shift?

1.11. Exercises

1. Find one recent (last 12 months) news article or company disclosure describing the use of data analytics or AI in an accounting, audit, or finance context. Write a short (150-word) summary of what the technology does and what problem it solves.
2. Look up the current AICPA CPA Exam Blueprint for the Business Analysis and Reporting (BAR) discipline. Identify two specific data-analytics-related skills listed and briefly explain, in your own words, what they mean.

2. Python and Quarto Environment Setup

i Learning Objectives

By the end of this chapter, you should be able to:

- Install Python, Quarto, and a code editor on your own computer
- Explain the difference between a Jupyter notebook (`.ipynb`), a plain Python script (`.py`), and a Quarto document (`.qmd`)
- Create, edit, and render a Quarto document containing Python code
- Render a Quarto book to both HTML and PDF
- Install and import the Python packages used throughout this book (pandas, numpy, matplotlib)

2.1. Why This Chapter Matters

Every chapter after this one assumes you can do three things: write Python code, run it, and see the results in a nicely formatted document. This chapter gets that pipeline working on your own laptop, once, so that the rest of the course is about *accounting analytics* rather than *fighting with software installation*.

2. Python and Quarto Environment Setup

If you get stuck at any point in this chapter, don't worry — environment setup is almost always the hardest part of a data analytics course, and it only gets easier from here.

2.2. The Three Tools You Need

Tool	What it does	Analogy
Python	The programming language that does the actual data work	The engine
Quarto	Combines your writing, Python code, and output into one document (HTML, PDF, Word)	The publishing system
A Code Editor (Also called Integrated Development Environment (IDE))	Where you write and run your code (this book uses Positron, but VS Code works identically)	The workshop

You need all three installed before you can complete the exercises in this book.

2.3. Step 1: Install Python

You can install the python from <https://www.python.org/downloads/>. We recommend you use this link to install a specific version of python. Your professor might recommend a specific version that you should use. If you use macOS, please click macOS to install python suitable for mac.

2.4. Step 2: Install Quarto

In addition, you can also install Python via the **Anaconda** or **Miniconda** distribution, because it comes bundled with the data packages we'll use (pandas, numpy, matplotlib) and makes managing versions easier than a bare Python install.

1. Download Miniconda from <https://docs.conda.io/en/latest/miniconda.html> (choose the installer for your operating system).
2. Run the installer with default settings.
3. Confirm the install worked by opening a terminal and typing:

```
python --version
```

You should see something like `Python 3.11.x` (the exact version doesn't need to match this).

Connect to Practice

If your future employer uses a specific data platform (e.g., a cloud data warehouse, or a company-managed Python environment), the *concepts* you learn here transfer directly — only the installation step changes.

2.4. Step 2: Install Quarto

1. Go to <https://quarto.org/docs/get-started/> and download the installer for your operating system.
2. Run the installer with default settings.
3. Confirm the install worked:

```
quarto --version
```

2. Python and Quarto Environment Setup

Since we'll render to PDF as well as HTML, also install a LaTeX distribution — Quarto includes a lightweight one you can install with a single command:

```
quarto install tinytex
```

This only needs to be done once per computer.

2.5. Step 3: Install a Code Editor

This book's examples use **Positron**, which can be downloaded from <https://positron.posit.co/download.html>. Positron is a code editor built specifically for data science that will feel very familiar if you've used VS Code before — the two share the same underlying editing engine, so keyboard shortcuts, extensions, and layout all work the same way. (If your instructor prefers plain VS Code, everything in this chapter applies identically.)

Figure 2.1 shows the general layout you'll be working in for the rest of this book: a file explorer on the left, an editor pane in the middle where you write your `.qmd` files, a terminal at the bottom where you run commands like `quarto render`, and a Source Control panel for Git (covered separately if your instructor uses version control in this course).

2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

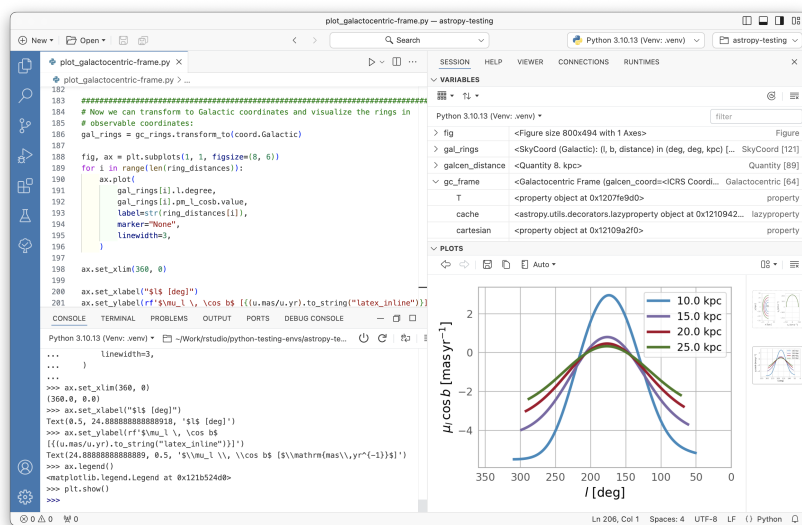


Figure 2.1.: A typical code-editor layout: file explorer, editor pane, session, and plots

Once installed, open your project folder in Positron/VS Code using **File** → **Open Folder**, and select the folder containing your `_quarto.yml` file.

2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

As you start doing accounting analytics work, you'll encounter three common file types for writing Python. It's worth understanding the difference early, since this book uses all three at different points.

2. Python and Quarto Environment Setup

2.6.1. 1. A Plain Python Script (.py)

A .py file contains only code — no narrative text, no formatted output. You run the whole file at once (or line by line) and see output printed to a terminal or console. This is the format used for production code, reusable functions, or automated jobs that don't need explanation baked in.

```
# analysis.py
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
print(df.head())
```

2.6.2. 2. A Jupyter Notebook (.ipynb)

A Jupyter notebook mixes code and narrative text in a sequence of **cells**, and shows output (tables, charts) directly underneath the code that produced it. This makes notebooks excellent for exploratory analysis — trying things out, looking at results immediately, and iterating.

Figure 2.2 shows a simplified notebook: a markdown cell explaining what's about to happen, a code cell that loads data, and the output (a data table) displayed directly below it.

2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

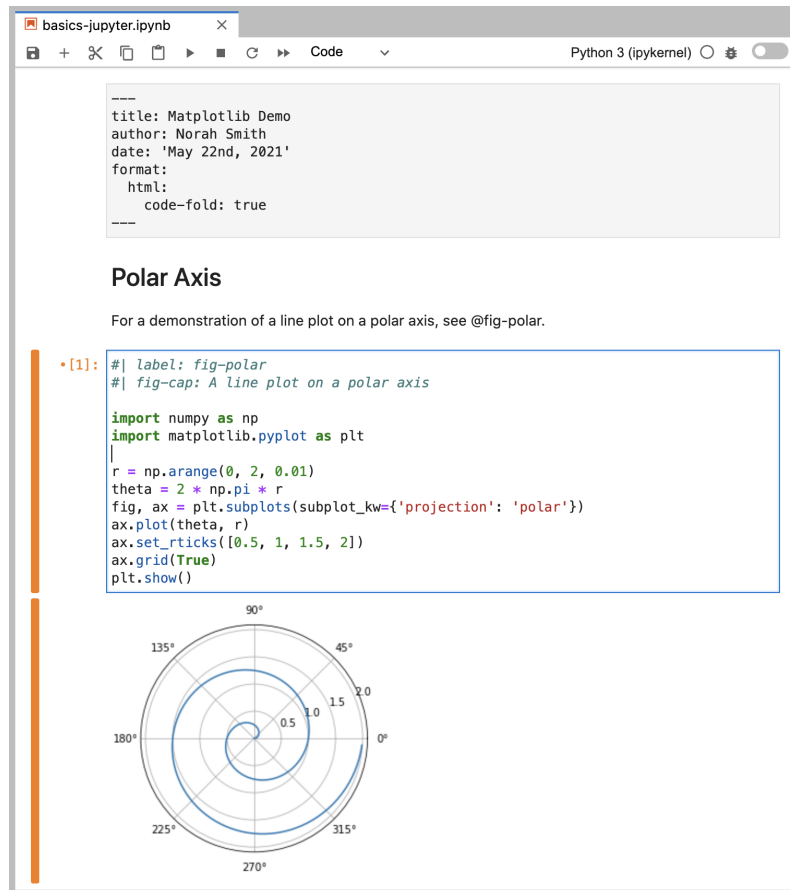


Figure 2.2.: A Jupyter notebook combines markdown cells, code cells, and their output in one scrollable document

You can open and run `.ipynb` files directly in Positron/VS Code, the same as in classic Jupyter.

2. Python and Quarto Environment Setup

2.6.3. 3. A Quarto Document (.qmd)

A Quarto document looks a lot like a notebook conceptually — narrative text plus embedded, runnable code — but it's a **plain text file** (easier to track with Git) that can render to multiple polished output formats: HTML, PDF, Word, and more. This is the format used for every chapter in this book.

A Quarto document has three parts:

1. A **YAML header** at the top (between `---` lines) with metadata like the title
2. **Narrative text** written in Markdown
3. **Code chunks**, marked with three backticks and curly braces

```
---
title: "Example Chapter"
---

## Analyzing the Trial Balance

Here we load the trial balance and preview the first few rows.

::: {.cell execution_count=1}
``` {.python .cell-code}
import pandas as pd

df = pd.read_csv("data/trial_balance.csv")
df.head()
```

::: {.cell-output .cell-output-display execution_count=1}
```{=html}
<div>
```



## 2.6. Three Ways to Write Python: Script, Notebook, or Quarto Document

```
<style scoped>
 .dataframe tbody tr th:only-of-type {
 vertical-align: middle;
 }

 .dataframe tbody tr th {
 vertical-align: top;
 }

 .dataframe thead th {
 text-align: right;
 }
</style>
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>account</th>
 <th>account_type</th>
 <th>debit</th>
 <th>credit</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>Cash</td>
 <td>Asset</td>
 <td>12500</td>
 <td>0</td>
 </tr>
 <tr>
 <th>1</th>
```

## 2. Python and Quarto Environment Setup

```
<td>Accounts Receivable</td>
<td>Asset</td>
<td>8200</td>
<td>0</td>
</tr>
<tr>
<th>2</th>
<td>Inventory</td>
<td>Asset</td>
<td>15300</td>
<td>0</td>
</tr>
<tr>
<th>3</th>
<td>Prepaid Insurance</td>
<td>Asset</td>
<td>1200</td>
<td>0</td>
</tr>
<tr>
<th>4</th>
<td>Equipment</td>
<td>Asset</td>
<td>42000</td>
<td>0</td>
</tr>
</tbody>
</table>
</div>
...
:::
:::
```

## 2.7. Your First Quarto Document

The output above shows account balances before adjusting entries.

! Which format does this book use?

Every chapter in *this* book is written as a `.qmd` file, since we want a single, polished, professional document (in both HTML and PDF) at the end of the semester — not a folder of separate notebooks. However, your instructor may ask you to *practice and experiment* in a Jupyter notebook before copying finished code into your chapter or assignment `.qmd` file. Think of the notebook as your scratch paper and the `.qmd` file as your final submission.

## 2.7. Your First Quarto Document

Let's create a small Quarto document from scratch to make sure everything works end to end.

**1. Create a new file** called `test.qmd` in your project folder with the following content:

```

title: "My First Quarto Document"
format: html

Testing My Setup

::: {.cell execution_count=2}
``` {.python .cell-code}
import pandas as pd
```

2. Python and Quarto Environment Setup

```
data = {"account": ["Cash", "Accounts Receivable", "Inventory"],
        "balance": [12500, 8200, 15300]}
```

```
df = pd.DataFrame(data)
df
---
```

```
::: {.cell-output .cell-output-display execution_count=2}
```{=html}
<div>
<style scoped>
 .dataframe tbody tr th:only-of-type {
 vertical-align: middle;
 }

 .dataframe tbody tr th {
 vertical-align: top;
 }

 .dataframe thead th {
 text-align: right;
 }
</style>
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>account</th>
 <th>balance</th>
 </tr>
 </thead>
 <tbody>
 <tr>
```

## 2.7. Your First Quarto Document

```
<th>0</th>
<td>Cash</td>
<td>12500</td>
</tr>
<tr>
<th>1</th>
<td>Accounts Receivable</td>
<td>8200</td>
</tr>
<tr>
<th>2</th>
<td>Inventory</td>
<td>15300</td>
</tr>
</tbody>
</table>
</div>
...

:::
:::
```

If you can see a table above with three accounts, your Python and Quarto setup is working correctly.

**2. Render it.** In the terminal, run:

```
quarto render test.qmd
```

**3. Open the result.** This creates `test.html` in the same folder — open it in your web browser and confirm you can see the table.

If this worked, your environment is fully set up. If you got an error, check the Troubleshooting section at the end of this chapter before moving on.

## 2. Python and Quarto Environment Setup

### 2.8. Rendering This Book's Project (HTML and PDF)

Your course textbook project is set up as a **Quarto book** (multiple chapters combined into one publication), which is slightly different from rendering a single file. From the terminal, inside your project folder (the one containing `_quarto.yml`):

```
quarto render
```

This single command renders *every* chapter listed in `_quarto.yml` and produces both formats we configured:

- `_book/index.html` — an HTML website you can open in a browser or host online
- a single PDF file inside `_book/` — a downloadable, printable version of the entire book

Figure 2.3 summarizes this process: your `.qmd` source files go in, **quarto render** executes the Python code and assembles the document, and both an HTML and a PDF version come out the other side from the *same source file* — you never have to write your content twice.

## 2.9. Installing the Python Packages You'll Need

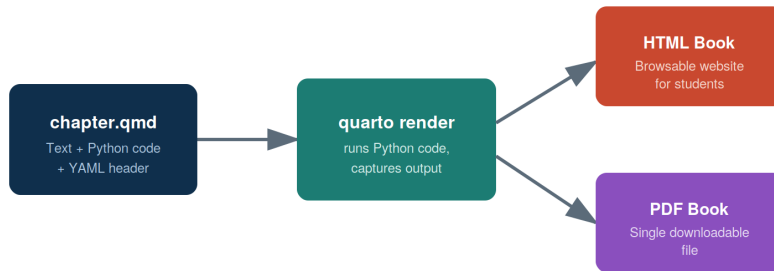


Figure 2.3.: The Quarto rendering workflow: one source document, two polished output formats

To preview your book live while editing (auto-refreshing as you save changes), use:

```
quarto preview
```

## 2.9. Installing the Python Packages You'll Need

This book relies on a small set of Python packages. If you installed Python via Anaconda/Miniconda, most of these are likely already available. To install (or confirm) them, run:

```
pip install pandas numpy matplotlib jupyter
```

Confirm they're installed correctly by running this in a `.qmd` code chunk or a notebook cell:

## 2. Python and Quarto Environment Setup

```
```{python}
import pandas as pd
import numpy as np
import matplotlib

print(pd.__version__)
print(np.__version__)
print(matplotlib.__version__)
```
```

If this runs without errors and prints three version numbers, you're ready for Chapter 3.

### 2.10. Organizing Your Project Folder

As your book (or your homework project) grows, keeping files organized will save you real time. Figure 2.4 shows the recommended structure, which matches how this book's own project folder is organized: one `.qmd` file per chapter, a shared `images/` folder for figures, and the configuration file (`_quarto.yml`) at the top level.



## 2.10. Organizing Your Project Folder

```
accounting-analytics-book/
 _quarto.yml
 index.qmd
 01-introduction-to-accounting-analytics.qmd
 02-python-quarto-setup.qmd
 03-data-fundamentals.qmd
 ... (remaining chapter files)
 references.qmd
 references.bib
 appendix-a-python-primer.qmd
 appendix-b-financial-statement-basics.qmd
 images/
 quarto-workflow.svg
 editor-layout.svg
 jupyter-notebook-mockup.svg
```

Figure 2.4.: Recommended project folder structure: one file per chapter, plus a shared images folder

A couple of habits worth building now:

- Keep all figures in a single **images/** folder rather than scattering them across the project — it makes the project easier to navigate and easier to back up.
- Give files and images descriptive, lowercase, hyphenated names (e.g., **revenue-by-quarter.png**, not **Figure1\_FINAL\_v2.png**) — this avoids broken links when you rename things later and matches conventions used in real data projects.

## 2. Python and Quarto Environment Setup

### 2.11. Troubleshooting Common Setup Issues

#### “quarto: command not found”

This usually means Quarto wasn’t added to your system’s PATH during installation. Try closing and reopening your terminal (and your code editor) after installing Quarto — this alone fixes the issue most of the time. If it persists, reinstall Quarto and confirm you allowed it to modify your PATH during setup.

#### “ModuleNotFoundError: No module named ‘pandas’”

Python can’t find the package because it isn’t installed in the environment your editor is using. Run `pip install pandas` in the same terminal your editor uses, and confirm you’re using the Python interpreter you expect (in Positron/VS Code, check the interpreter shown in the bottom status bar).

#### PDF rendering fails but HTML works fine

This almost always means the LaTeX installation is missing or incomplete. Run `quarto install tinytex` and try again. If it still fails, re-render with `quarto render --to pdf --log render.log` and check the log file for the specific missing package.

### 2.12. Chapter Summary

- Python, Quarto, and a code editor (Positron or VS Code) are the three tools needed for this course, and each is installed once at the start of the semester.

- A `.py` script, a `.ipynb` notebook, and a `.qmd` Quarto document all run Python code, but serve different purposes: scripts for production code, notebooks for exploration, and Quarto documents for polished, reproducible reports — which is why this book is written entirely in `.qmd` files.
- `quarto render` turns your `.qmd` source into both HTML and PDF from a single source of truth.
- Keeping a consistent project folder structure (chapters at the top level, a shared `images/` folder) will save you time throughout the semester.

## 2.13. Exercises

1. Complete the “Your First Quarto Document” walkthrough in this chapter. Take a screenshot of your rendered HTML output showing the table, and submit it as instructed by your course syllabus.
2. In your own words (2–3 sentences), explain when you would choose to use a Jupyter notebook versus a Quarto document for a piece of analytical work.
3. Intentionally introduce one small error into your `test.qmd` file (e.g., misspell `pandas` as `panadas`), render it, and copy the error message you get. Then fix it and re-render successfully. This exercise is designed to make error messages feel less intimidating early in the course.



## 3. Data Fundamentals for Accountants

### **i** Learning Objectives

By the end of this chapter, you should be able to:

- Explain tidy data principles and identify whether a table is tidy or messy
- Identify the four levels of measurement and match them to common accounting variables
- Recognize the main data types encountered in accounting data (numeric, categorical, date/time, text, boolean)
- Describe the structure and relationship between the chart of accounts, general ledger, trial balance, and subledgers
- Explain the difference between transaction-level and summarized data, and why analytics often needs the former
- Identify common data quality issues in real-world accounting datasets

### 3.1. Why Data Structure Matters Before You Start Coding

It's tempting to jump straight into pandas and start writing code — and we will, starting next chapter. But most real-world accounting analytics

### 3. Data Fundamentals for Accountants

problems are **data problems**, not coding problems. A well-written line of Python code applied to poorly structured or misunderstood data will still give you a wrong (or misleading) answer.

This chapter builds the conceptual foundation you need before touching code: what “well-structured” data looks like, what kinds of data you’ll encounter, how core accounting datasets are organized, and what tends to go wrong with real data. Think of this as learning to read a map before you start driving.

## 3.2. Tidy Data Principles

**Tidy data** is a simple, widely-used standard for organizing tabular data so that it’s easy to analyze. A dataset is tidy when:

1. **Each variable forms a column.**
2. **Each observation forms a row.**
3. **Each type of observational unit forms a table.**

This sounds abstract, so let’s look at an example. Suppose you have quarterly revenue by product line.

**Messy (wide) version** — each quarter is its own column:

| product_line | Q1     | Q2     | Q3     | Q4     |
|--------------|--------|--------|--------|--------|
| Consulting   | 42,000 | 45,500 | 47,200 | 50,100 |
| Software     | 61,000 | 58,000 | 63,500 | 70,200 |

This looks fine to read as a human, and it’s a completely normal way to present numbers in a report. But it’s **not tidy**: “quarter” is a variable that’s been spread across four columns instead of living in a single column.

**Tidy (long) version** — the same data, restructured:

### 3.3. Levels of Measurement

| product_line | quarter | revenue |
|--------------|---------|---------|
| Consulting   | Q1      | 42,000  |
| Consulting   | Q2      | 45,500  |
| Consulting   | Q3      | 47,200  |
| Consulting   | Q4      | 50,100  |
| Software     | Q1      | 61,000  |
| Software     | Q2      | 58,000  |
| Software     | Q3      | 63,500  |
| Software     | Q4      | 70,200  |

#### Connect to Practice

Tidy data isn't just an academic preference. In Chapter 4, you'll see that pandas operations — filtering, grouping, plotting — are dramatically easier when your data is tidy. A wide table like the “messy” example above requires extra reshaping steps before you can, say, plot revenue trends across all product lines on one chart.

You don't need to memorize a rulebook here — the practical test is: *if I wanted to filter, group, or plot by any single variable, is that variable sitting in one clearly labeled column?* If yes, you're likely looking at tidy data.

## 3.3. Levels of Measurement

Before you calculate anything — an average, a sum, a comparison — it's worth asking: *what kind of variable is this, and what operations actually make sense on it?* This is the idea behind **levels of measurement**, a classification used throughout statistics.

There are four levels, each allowing more mathematical operations than the last:

### 3. Data Fundamentals for Accountants

| Level           | Description                                                          | Accounting Example                                               | Meaningful Operations                                 |
|-----------------|----------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------|
| <b>Nominal</b>  | Categories with no inherent order                                    | Account type (Asset, Liability, Equity), Department, Vendor name | Count, mode, group by                                 |
| <b>Ordinal</b>  | Ordered categories, but differences between levels aren't meaningful | Credit risk rating (Low/Medium/High), Audit opinion severity     | Count, mode, median, ranking                          |
| <b>Interval</b> | Meaningful differences, but no true zero                             | Fiscal year, Credit score (in some scales)                       | Addition/subtraction, mean, but not meaningful ratios |
| <b>Ratio</b>    | True zero point; all arithmetic is meaningful                        | Account balance, Transaction amount, Units sold                  | Mean, sum, ratios, all standard arithmetic            |

#### ! Why This Matters in Practice

It's easy to make a technically-valid-looking calculation that is conceptually meaningless. For example: averaging a customer's credit risk rating of "Low, Medium, High" by assigning them 1, 2, 3 and computing a mean of 1.8 *looks* like a real number, but "1.8" doesn't correspond to any real risk category — ordinal data doesn't support that kind of averaging. Similarly, summing account numbers from your chart of accounts (nominal data, even though it's stored as a



number) produces a number with no accounting meaning at all. We'll return to this idea in Chapter 5, where choosing the right summary statistic depends directly on a variable's level of measurement.

## 3.4. Data Types You'll Encounter

Related to levels of measurement, but more about how data is *stored and represented* in a dataset, are data types. In accounting data, you'll regularly see:

- **Numeric — continuous:** dollar amounts (account balances, transaction amounts) — ratio-level, all arithmetic makes sense
- **Numeric — discrete:** counts (units sold, number of invoices) — ratio-level, but only whole numbers make sense
- **Categorical:** account type, department, cost center, vendor — typically nominal, sometimes ordinal (e.g., risk tier)
- **Date/time:** transaction date, fiscal period, posting date — deserves special attention, covered next
- **Text/string:** transaction descriptions, memo fields, vendor names — often messy and inconsistently formatted
- **Boolean/flag:** `is_reversed`, `is_intercompany`, `is_reconciled` — true/false indicators, nominal with only two categories

### 3.4.1. A Special Note on Dates

Dates cause more headaches in accounting data than almost any other data type, for a few reasons:

- They can be stored as **text** ("01/15/2025") instead of an actual date type, which prevents sorting and date arithmetic from working correctly

### 3. Data Fundamentals for Accountants

- **Format ambiguity:** is 03/04/2025 March 4th or April 3rd? This depends on regional conventions (US vs. most of the rest of the world) and is a common source of silent errors
- **Fiscal year vs. calendar year:** a company with a fiscal year ending June 30 will label transactions in “FY2025” that occurred in *calendar* 2024

We’ll handle dates properly with pandas in Chapter 4 — for now, just know that a date column that “looks fine” when you eyeball a spreadsheet is one of the most common sources of hidden bugs in accounting analytics.

## 3.5. Core Accounting Data Structures

Most accounting analytics work touches one or more of four related data structures. Figure 3.1 shows how they connect.

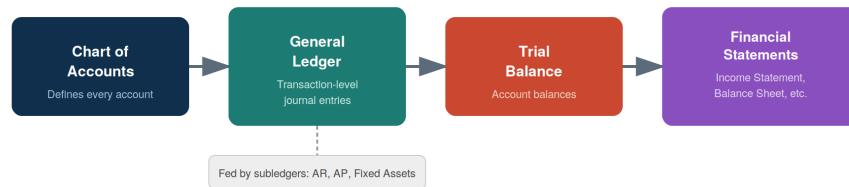


Figure 3.1.: How core accounting data structures relate: the chart of accounts defines every account, transactions flow through the general ledger (fed by subledgers), and roll up into a trial balance and ultimately the financial statements

### 3.5. Core Accounting Data Structures

#### 3.5.1. Chart of Accounts (COA)

The chart of accounts is the master list of every account a company uses — essentially a data dictionary for the rest of the accounting system. A typical COA entry includes an account number, account name, and account type (Asset, Liability, Equity, Revenue, Expense).

#### 3.5.2. General Ledger (GL)

The general ledger contains **transaction-level detail** — every individual journal entry, each with a debit and a credit side, a date, a description, and (typically) the account(s) affected. This is the most granular, information-rich accounting dataset, and the one most audit and fraud analytics work is performed on.

#### 3.5.3. Subsidiary Ledgers

Subsidiary ledgers (Accounts Receivable, Accounts Payable, Fixed Assets, Payroll, etc.) hold detailed records for a specific area that then roll up into summary entries in the general ledger. For example, the AR subledger tracks every individual customer invoice and payment, but the GL might only see a periodic summary entry.

#### 3.5.4. Trial Balance

The trial balance is a **summarized snapshot**: the ending balance of every account in the chart of accounts, as of a point in time. This is the level of detail you worked with in Chapter 2 — one row per account, not one row per transaction. A trial balance is a checkpoint used to confirm that total debits equal total credits before financial statements are prepared.

### 3.6. Transaction-Level vs. Summarized Data

This distinction matters more than it might first appear:

- A **trial balance** can look completely normal — accounts balanced, nothing unusual — while individual **transactions** underneath it hide errors, unusual patterns, or fraud.
- For example, imagine a “Travel Expense” account with a perfectly reasonable ending balance of \$45,000 for the year. That summary number gives no indication that \$38,000 of it came from a single suspicious transaction posted on a Sunday by an employee who doesn’t normally have travel expenses.

This is exactly why audit analytics (Chapter 8) and fraud detection (Chapter 9) typically require **transaction-level general ledger data**, not just trial balances — the interesting patterns often only show up at the most granular level.

### 3.7. Common Data Quality Issues in Accounting Data

Real accounting data is rarely clean. Some of the most common issues you’ll encounter:

- **Missing values:** blank department codes, missing dates, unpopulated memo fields
- **Duplicate entries:** the same transaction entered twice, sometimes with slightly different formatting
- **Inconsistent formatting:** dates stored as text in different formats, currency values stored with \$ symbols or commas (which prevents numeric calculations until cleaned), inconsistent capitalization in account or vendor names (“Cash” vs. “cash” vs. “CASH ” with trailing whitespace)

### 3.8. A Preview Look at a Real Dataset

- **Inconsistent categorical labels:** the same department entered as "HR", "H.R.", and "Human Resources" across different rows
- **Outliers vs. errors:** an unusually large transaction might be a legitimate one-time event (a building purchase) or a data entry error (an extra zero) — these require judgment, not just a statistical rule, to tell apart

#### A Realistic Messy Example

Imagine you received this general ledger excerpt exactly as extracted from a client's system:

| entry_date | account | debit      | department |
|------------|---------|------------|------------|
| 01/15/2025 | Cash    | \$4,500.00 | Sales      |
| 1-15-2025  | cash    | 4500       | sales      |
| 01/15/2025 | Cash    | 4,500      |            |
| 01/32/2025 | Cash    | 45,000     | Sales      |

This tiny sample already contains: inconsistent date formats, inconsistent account name capitalization, inconsistent currency formatting, a missing department value, a likely duplicate entry, an impossible date (01/32/2025), and a value that may be a data-entry error (an extra zero turning \$4,500 into \$45,000). Spotting these issues *before* you calculate anything is a core skill — we'll clean data like this hands-on in Chapter 4.

### 3.8. A Preview Look at a Real Dataset

To connect these ideas to something concrete, look at the sample transaction-level general ledger below (`general_ledger_sample.csv`), covering two weeks of journal entries for a small company. Each journal

### 3. Data Fundamentals for Accountants

entry (`entry_id`) has at least two rows — one debit, one credit — matching how double-entry accounting actually works.

| en-try_id | en-try_date | ac-count               | ac-count_type | de-scrip-tion                | debit | credit | de-part-ment | posted_by |
|-----------|-------------|------------------------|---------------|------------------------------|-------|--------|--------------|-----------|
| JE1001    | 2025-01-03  | Cash                   | Asset         | Cus-tomer pay-ment re-ceived | 4500  | 0      | Sales        | jsmith    |
| JE1001    | 2025-01-03  | Ac-counts Re-ceiv-able | Asset         | Cus-tomer pay-ment re-ceived | 0     | 4500   | Sales        | jsmith    |
| JE1002    | 2025-01-05  | Inven-tory             | Asset         | Pur-chase of mer-chan-dise   | 7200  | 0      | Oper-ations  | mchen     |
| JE1002    | 2025-01-05  | Ac-counts Payable      | Lia-bility    | Pur-chase of mer-chan-dise   | 0     | 7200   | Oper-ations  | mchen     |

A few things worth noticing just from visual inspection, before we write a single line of pandas code in Chapter 4:

### 3.9. Chapter Summary

- This table is **tidy**: each column is a single variable (date, account, debit, credit, etc.), and each row is one line of a journal entry.
- The **level of measurement** varies by column: **debit/credit** are ratio-level (arithmetic is meaningful), **account\_type** and **department** are nominal (categories only), **entry\_date** is a date type requiring careful handling.
- This is **transaction-level** data — far more granular than the trial balance from Chapter 2 — which is exactly the kind of dataset audit and fraud analytics techniques (Chapters 8–9) are designed to work with.

We'll load and work with this exact file in Chapter 4.

## 3.9. Chapter Summary

- Tidy data — one variable per column, one observation per row, one observational unit per table — makes analysis dramatically easier and is the standard this book (and pandas) assumes.
- Levels of measurement (nominal, ordinal, interval, ratio) determine which calculations are actually meaningful for a given variable — averaging an ordinal rating or summing a nominal code produces numbers without real meaning.
- Accounting data typically falls into a few types: numeric, categorical, date/time, text, and boolean — each with its own quirks, dates especially.
- The chart of accounts, general ledger, subledgers, and trial balance are related but distinct structures, moving from transaction-level detail to summarized snapshots.
- Analytics work — especially audit and fraud analytics — usually requires transaction-level data, since summarized data can hide the patterns that matter most.

### 3. Data Fundamentals for Accountants

- Real accounting data commonly has quality issues: missing values, duplicates, inconsistent formatting, and ambiguous outliers, all of which need to be identified before analysis begins.

## 3.10. Discussion Questions

1. Look back at the “wide” quarterly revenue example early in this chapter. Why might a company’s finance team *prefer* the wide format for a printed report, even though it’s not tidy for analysis purposes?
2. Give one example (not from this chapter) of an accounting variable at each level of measurement: nominal, ordinal, interval, and ratio.
3. Why might a company’s trial balance appear completely normal in a given month, even if fraudulent transactions occurred during that period?

## 3.11. Exercises

1. Take the “messy” wide revenue table from this chapter and manually rewrite it in tidy (long) format on paper or in a spreadsheet.
2. Using the messy general ledger excerpt in the callout box above, list every data quality issue you can find, and for each one, briefly describe how you would investigate or resolve it if this were real client data.
3. Download `general_ledger_sample.csv` (provided alongside this chapter) and open it in a spreadsheet program. Identify: (a) how many unique journal entries it contains, (b) how many unique departments appear, and (c) whether total debits equal total credits.



**Part II.**

## **Part II: Core Data Skills**



## **4. Data Wrangling with Pandas**



## **5. Exploratory Data Analysis for Financial Data**



## **6. Data Visualization for Accounting**





**Part III.**

**Part III: Applied Accounting  
Analytics**



## **7. Financial Statement Analysis with Python**



## **8. Audit Analytics**



## **9. Fraud Detection and Anomaly Detection**





## **10. Revenue Recognition and Transaction-Level Analysis**



## **11. Tax Analytics**



**Part IV.**

**Part IV: Predictive Methods  
and GenAI**



## **12. Introduction to Predictive Modeling**





## **13. Generative AI in Accounting**



**Part V.**

**Part V: Communication and  
Capstone**



## **14. Automated Reporting and Dashboards with Quarto**



## **15. Data Ethics, Governance, and Reproducibility**





## **16. Capstone Project**



## References



## **A. A Primer on Python**



## **B. Financial Statement Basics**





## **C. An Introduction to EDGAR Database**

